Mansi Davkhar

D15C/24

MLDL Experiment 8

Aim: To design and train a Convolutional Neural Network (CNN) on image datasets (e.g., MNIST/CIFAR).

---

1. Dataset Source

Source: Keras Datasets - CIFAR10

2. Dataset Description

- **Classes:** 10 (Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship, Truck).
- **Training Set:** 50,000 color images.
- **Test Set:** 10,000 color images.
- **Image Size:** 32 * 32 pixels, 3 color channels (RGB).

3. Mathematical Formulation

A CNN uses **Convolution ($*$)** to detect features:

$$(I * K)_{x,y} = \sum_i \sum_j I_{x+i,y+j} \cdot K_{i,j}$$

The **ReLU** activation $f(x) = \max(0, x)$ introduces non-linearity, and **Softmax** provides the final probability for each class.

4. Algorithm Limitations

- **Resolution:** 32 * 32 is very small; the model may struggle with fine details (like the difference between a cat and a dog's ears).
- **Hardware:** Requires a GPU (in Colab: Edit > Notebook Settings > Hardware Accelerator > T4 GPU) for reasonable training times.

5. Methodology / Workflow

1. **Data Loading:** Use datasets.cifar10.load_data().
2. **Normalization:** Divide pixel values (0-255) by 255.0 to get a range of [0, 1].
3. **Architecture:** * 3 Convolutional layers (increasing filters: 32 -> 64 -> 64).
    - Max-Pooling after each layer to reduce spatial size.
    - Flatten into a 1D vector.
    - Dense layers for final classification.
4. **Training:** 10 epochs using the 'Adam' optimizer.

6. Code

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt

# 1. LOAD DATA (No upload needed!)
(train_images, train_labels), (test_images, test_labels) = datasets.cifar10.load_data()
# 2. PREPROCESS
# Normalize pixel values to be between 0 and 1
train_images, test_images = train_images / 255.0, test_images / 255.0
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']
# 3. BUILD THE CNN
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10) # 10 classes
])
# 4. COMPILE
model.compile(optimizer='adam',
```

```
        loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
        metrics=['accuracy'])
# 5. TRAIN
# We do 10 epochs. It should take about 1-2 mins on Colab GPU.
history = model.fit(train_images, train_labels, epochs=10,
                    validation_data=(test_images, test_labels))
# 6. EVALUATE
test_loss, test_acc = model.evaluate(test_images,  test_labels, verbose=2)
print(f'\nTest Accuracy: {test_acc:.4f}')
# 7. VISUALIZE A PREDICTION
plt.figure(figsize=(2,2))
plt.imshow(test_images[0])
plt.title(f"Pred: {class_names[tf.argmax(model.predict(test_images[0:1])[0])]}")
plt.axis('off')
plt.show()
```

Output:

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 12s 0us/step
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `inpu
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 75s 47ms/step - accuracy: 0.4560 - loss: 1.5041 - val_accuracy: 0.5584 - val_loss: 1.2379
Epoch 2/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 69s 44ms/step - accuracy: 0.5967 - loss: 1.1377 - val_accuracy: 0.6182 - val_loss: 1.0704
Epoch 3/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 68s 44ms/step - accuracy: 0.6544 - loss: 0.9851 - val_accuracy: 0.6564 - val_loss: 0.9812
Epoch 4/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 76s 49ms/step - accuracy: 0.6934 - loss: 0.8785 - val_accuracy: 0.6710 - val_loss: 0.9353
Epoch 5/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 69s 44ms/step - accuracy: 0.7174 - loss: 0.8048 - val_accuracy: 0.6871 - val_loss: 0.8972
Epoch 6/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 74s 47ms/step - accuracy: 0.7391 - loss: 0.7423 - val_accuracy: 0.6890 - val_loss: 0.9035
Epoch 7/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 70s 45ms/step - accuracy: 0.7587 - loss: 0.6924 - val_accuracy: 0.6972 - val_loss: 0.8830
Epoch 8/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 86s 47ms/step - accuracy: 0.7738 - loss: 0.6449 - val_accuracy: 0.7000 - val_loss: 0.8954
Epoch 9/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 75s 48ms/step - accuracy: 0.7871 - loss: 0.6065 - val_accuracy: 0.7187 - val_loss: 0.8466
Epoch 10/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 72s 42ms/step - accuracy: 0.8005 - loss: 0.5712 - val_accuracy: 0.7070 - val_loss: 0.9013
313/313 - 3s - 11ms/step - accuracy: 0.7070 - loss: 0.9013

Test Accuracy: 0.7070
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 103ms/step
```

7. Performance Analysis

The model's performance was monitored over 10 epochs, and the following observations were made:

- **Accuracy:** The training accuracy reached **80.05%**, while the validation (test) accuracy settled at **70.70%**. This indicates the model is learning the visual patterns of the 10 classes effectively.
- **Loss Convergence:** The training loss decreased steadily from **1.5041** to **0.5712**. This downward trend confirms that the Adam optimizer and the backpropagation algorithm were successfully minimizing errors.
- **Generalization Gap:** There is a roughly **10% gap** between training accuracy (80%) and test accuracy (70%). This suggest a slight amount of **overfitting**, where the model is starting to memorize specific details of the training images rather than learning universal features.
- **Computational Efficiency:** On average, the model took about **72–75 seconds per epoch**, demonstrating the intensive computational nature of processing 50,000 color images through multiple convolutional layers.

8. Applications

The CNN architecture used in this experiment is the foundation for various real-world technologies:

- **Autonomous Vehicles:** Identifying objects like pedestrians, traffic lights, and other cars in real-time.
- **Facial Recognition:** Powering security systems and social media "tagging" features.
- **Medical Diagnostics:** Analyzing X-rays or MRI scans to detect tumors or abnormalities with higher precision than the human eye.
- **Content Moderation:** Automatically identifying and filtering inappropriate visual content on social media platforms.

9. Conclusion

The experiment successfully demonstrated the design and implementation of a **Convolutional Neural Network** for image classification. By using the CIFAR-10 dataset, we moved beyond simple grayscale digits to complex, multi-channel color images.

Key takeaways include:

1. **Spatial Intelligence:** Unlike the standard ANN used in the first experiment, the CNN's convolutional layers successfully preserved the spatial structure of images, allowing it to achieve a high accuracy of **70.70%**.
2. **Architecture Impact:** The use of multiple layers (Conv2D and MaxPooling) allowed the model to build a hierarchy of features—from simple edges in the first layer to complex object shapes in the final layers.
3. **Future Improvements:** To bridge the generalization gap and reach >85% accuracy, future experiments could incorporate **Data Augmentation** (flipping/rotating images) and **Dropout layers** to further penalize overfitting.